DroidKit

Help, Setup & Policies Guide

The free, honest Android + repair toolkit

DOABLE | CONDITIONAL | NOT-BY-SOFTWARE | UNVERIFIED

"No fake 100%. Bands, not promises - physics decides, we just report it."

Version 1.1.0 | Full-colour edition, August 2026
Publisher: AISACTECH | Free & open source (MIT)

Install | Setup | Every tool | Policies | FAQ

https://github.com/AISACTECH/droidkitv1
Works offline. Nothing phones home. Your devices only.

What's inside

Read it like a traffic light: the same colour bands used everywhere in the app run through every page of this guide. Green means physics allows it, amber means it depends (model, firmware, or a named third party), red means nobody's software can do it - and grey means we have not bench-verified it yet.

- 1 The policies - promises, rules, refusals ..... page 2
- 2 Setup, in detail - from download to first scan ..... page 4
- 3 Every tool in the app, explained ..... page 6
- 4 Myth vs physics - the two cards that stop scams ..... page 9
- 5 Questions & answers (the counter favourites) ..... page 10
- 6 The bands, and the words behind them ..... page 13
- 7 Getting human help ..... page 15

How to use this guide

1. In a hurry at the counter? Section 2 (Setup) and Section 5 (FAQ) answer 90% of questions.
2. Before any repair job, read the matching Policy in Section 1 and the band colour in Section 5.
3. This same content lives inside the app: sidebar -> Help & Info. The app version is searchable.
4. The printable counter cards in docs/kid-sheets/ are the one-page companions to this guide.

THE HONESTY LAW

No page of this guide claims what physics cannot deliver. Where a job needs a carrier, Google, a lender or a server, this guide says so - and names the honest route instead.

1 The policies - promises, rules, refusals

Nine plain-word policies that every feature in DroidKit obeys. They are colour-coded: green is a promise we make to you, amber is a rule that keeps you safe or legal, red is a permanent refusal.

1. The Honesty Law (our first and highest policy)

PROMISE

DroidKit never prints a success number it cannot prove. You will not find '100% FRP removal, Android 15/16' here, because that claim is false for every tool on Earth - the decision lives on Google's server, not in any cable or app. Anyone selling you that line is selling a scam.

Instead: traffic-light bands (DOABLE / CONDITIONAL / NOT-BY-SOFTWARE), named limits, named risks, and a bench log where real measurements - not marketing - calibrate every claim. Where we have not measured something, it is labelled UNVERIFIED in plain sight.

Where the math genuinely is deterministic (legacy Huawei modems), we say exactly that and show the real published examples the engine was verified against. Per-class truth is the product.

2. Your devices only - the ownership policy

RULE

Use DroidKit on devices YOU own, or a customer's device with their clear authorization. Every repair lane carries this consent rule on screen, and the in-app consent card (docs/kid-sheets/CONSENT-CARD.md) is free to print for shop counters.

FRP, screen locks, and passwords exist to protect owners from thieves. If you cannot sign in to the account that locked a device, our position is simple: the correct party is the account holder, the seller, or the platform's official recovery - not a bypass.

3. Jobs we refuse, permanently (and why)

NEVER

Lender/financed-phone MDM locks (Watu Simu, M-Kopa, Lipa Mdogo Mdogo and similar): a 'carrier-locked' phone bought on instalments in Kenya is almost always a LENDER lock, not a carrier lock. The phone belongs to the lender until the last payment. Defeating it is not unlock - it is taking property. The honest route: finish paying, or call the lender. We will not build this, ever.

IMEI rewriting: illegal in Kenya under Communications Authority rules and in most countries. No DroidKit feature touches the IMEI, by design.

Stolen or found devices: if ownership cannot be shown, the answer is the official-recovery page or the police - not a tool.

4. Privacy and your data

PROMISE

DroidKit runs on YOUR computer. There is no DroidKit account, no sign-up, and no DroidKit server that receives your device data. Device information, logs, and settings stay in local storage on your machine (the app's own store files).

The one thing that CAN leave your machine is the bench log - and only when you personally export the JSON file and choose to share it. Nothing uploads itself.

No adverts, no trackers, no bundled 'offers'. The codebase is open - the claim is falsifiable by reading it.

5. Free - the pricing policy

PROMISE

DroidKit is free and open source (MIT licence). No subscription, no trial clock, no 'credits', no feature locked behind a payment page inside the app.

Some jobs genuinely require third parties (for example a $3-8 NCK code service for ZTE/Alcatel MiFis, or an authorized Samsung/Google account recovery). We name them honestly, we take no cut, and we always show the free path first when one physically exists.

6. Lab features are EXPERIMENTAL

RULE

The FRP Lab (Patch Oracle) and the Rescue Lab are research benches. They are clearly badged EXPERIMENTAL in the app. They teach, predict, detect, and gate - and where software physically cannot act, they say so instead of pretending.

Predictions from the Patch Oracle are falsifiable: each forecast states what would prove it wrong, and the calibration record is kept in the open. Wrong predictions are corrected publicly, in the changelog.

7. Safety policy: counters, bricks, and your photos

RULE

Attempt counters are sacred. Modems and MiFis permanently hard-lock after too many wrong codes (some Alcatels die after 3). The app's Auto-Session reads remaining attempts BEFORE allowing any code entry, blocks at 2 or fewer, validates the IMEI checksum, and never lets a machine fire the entry command - a human clicks it.

Data loss is always named before a route is suggested. Every method card states if it ERASES the device. Fast, flashy and data-destroying is a bad trade we refuse to hide.

Brick risk is stated wherever flashing or boot-pin work is involved, with the model-exactness warning ('never follow a guide for a similar model').

8. Legality quick notes (Kenya first)

NOTE

Self-unlocking a device you OWN (carrier-unlocking your own MiFi or phone, removing your own forgotten password) is legal. Unlocking removes the lock, not physics - a 3G-era dongle will still never see Faiba's band-28 4G.

Illegal / off-limits: IMEI rewriting; defeating lender MDM (see policy 3); bypassing activation locks on devices whose ownership you cannot show.

When in doubt, the official carrier or platform route is printed in the app alongside the DIY route, so you can always choose the fully-official path.

9. Support policy and fair expectations

NOTE

Support happens in the open via GitHub issues - so answers help the next person too. Include your DroidKit version, your device model, what you clicked, and what you expected vs what happened.

We will never ask for your passwords, your Google/Samsung account, your unlock codes, or remote access to your computer. Anyone doing so in our name is an impostor.

Fixes land in the open repo. If a job is physically impossible, the honest answer will stay 'no' - with the best real route attached - rather than a pretend feature.

2 Setup, in detail - from download to first scan

Sixty seconds if all goes well. If it does not, the symptom doctor at the end of this section is the page to print and keep next to the computer.

Quick start

1. Install DroidKit on your computer (next page shows Windows, macOS and Linux).
2. On the phone: Settings -> About phone -> tap 'Build number' 7 times to unlock Developer options.
3. In Developer options, switch ON 'USB debugging'.
4. Connect the phone with a good USB cable (a DATA cable - some cheap cables are charge-only) and tap 'Allow' on the phone's USB-debugging prompt.
5. In DroidKit, open Devices and wait for your phone to appear, then click it to select it.
6. Open Help (this view) any time, and read the band colours before trusting any job: green, amber, red.

Install DroidKit

Get the installer for your computer from the project's GitHub Releases page (link on the last page).

1. Windows: download the .msi (or .exe) installer, double-click it, and follow the prompts. If SmartScreen warns because the app is new, click 'More info' -> 'Run anyway' (the binary is built publicly by GitHub Actions from this repo).
2. macOS: download the .dmg, open it, and drag DroidKit into Applications. On first open, right-click -> Open if Gatekeeper hesitates.
3. Linux: download the .AppImage (chmod +x it, then run) or the .deb (sudo dpkg -i file.deb).
4. Building from source instead: install Node 20+ and Rust, then run 'npm install' and 'npm run tauri:build'. The repo's docs/CI-GREEN-GUIDE.md explains the one-time automation setup for maintainers.

USB drivers (Windows only)

macOS and Linux normally need nothing. On Windows, the phone appears only after its ADB driver exists.

1. Install your phone brand's official driver (Samsung, Xiaomi, Transsion for Tecno/Infinix/Itel, etc.) - or Google's universal 'Google USB Driver' for Pixel-class devices.
2. In Device Manager, the phone should appear as an 'Android ADB Interface' (not 'Unknown device'). If it shows an error icon, update its driver manually and pick the ADB one.
3. Try a rear motherboard USB port and a different cable if installation loops - charge-only cables are the number one silent cause.

Switch on USB debugging (on the phone)

1. Settings -> About phone -> 'Build number': tap it 7 times until it says 'You are now a developer'. (Transsion phones: it can hide under About phone -> 'Build number' too; some show it after tapping 'Version'.)
2. Back in Settings -> System -> Developer options -> turn ON 'USB debugging'.
3. Plug into the computer. A prompt 'Allow USB debugging?' appears on the phone: tick 'Always allow from this computer' and tap Allow. If no prompt appears, unplug, run 'revoke USB debugging authorizations' in Developer options, and retry.
4. Wireless pairing (Android 11+): Developer options -> 'Wireless debugging' -> 'Pair device with pairing code', then use DroidKit's + button in the sidebar.

First use - the 60-second tour

1. Devices: pick your phone. Everything else unlocks after a device is selected - except Help, which always works.
2. System Info: confirms what is really inside (brand, model, Android version, chip). Every repair decision starts from these facts.
3. FRP Removal: the guided classic workflow for supported scenarios. FRP Lab: the experimental Patch Oracle - physics-layer survival and patch forecasts.
4. Rescue Lab: six repair lanes (PC password, carrier unlock, MiFi/modem, button phone, screen lock, black screen). Read a lane's band colour before starting any job.
5. Files / Apps / Screen / Logcat / Shell: everyday power tools once the phone is connected.

Symptom doctor - when it refuses

Symptom	Why it happens	The fix

Symptom	Why it happens	The fix
'npm install' or the app fails to install from GitHub	Old clone conflicted with generated files, or a syntax slip in an edited file	Delete the folder and clone fresh, then: npm install -> npm run lint -> npm run build. The paired-devices store syntax slip that broke CI was fixed; docs/CI-GREEN-GUIDE.md has the one-time bot setup.
Phone not listed in Devices	USB debugging off, missing driver, or charge-only cable	Follow the Setup page top to bottom; test with another cable first - it is the most common cause.
'Unauthorized' next to the device	The phone's Allow prompt was not accepted	Unplug, revoke USB debugging authorizations on the phone, replug, tick 'Always allow', tap Allow.
MiFi unlock code 'not accepted'	Wrong code generation for the model era, or counter already low	Stop. Read attempts left first (the Modem lane shows how). One correct-generation code, not guesses.
Feature worked in a YouTube video but not here	Many videos show server-side-impossible 'bypasses' or patched firmware holes	Check the band colour of your exact model in the Rescue Lab. NOT-BY-SOFTWARE means nobody's software does it - the honest route is printed.
The app feels slow to start	First-run antivirus scan of a new binary, or an old hard disk	Second launch is the fair one. Measured first-load wire size is about 260 kB gzip - see docs/PERFORMANCE.md.

3 Every tool in the app, explained

What each view in the sidebar does and the exact clicks to drive it. Remember: Help works without any device connected - everything else wakes up after you select a device.

Devices

TOOL

Your connected phones and tablets, USB or wireless.

1. Connect by cable (USB debugging on) or pair wirelessly (Android 11+) with the + button.
2. Click a device to select it - the other views wake up.
3. If nothing appears: cable first, driver second, debugging prompt third.

System Info

TOOL

The truth sheet: real model, Android version, security patch, chipset, battery.

1. Open it immediately after connecting - every rescue decision depends on these facts.
2. The patch level here feeds the Patch Oracle's survival estimate.

FRP Removal (classic)

TOOL

The guided FRP workflow covering several hundred known models and methods.

HONEST NOTE: Server-side FRP (Android 15/16 era) cannot be bypassed by any software, including this one - the app says so on screen instead of pretending.

1. Select the brand and exact model from the catalogue.
2. Follow the on-screen steps in order; skip nothing.
3. Results differ per model and patch level - that is physics, not a defect.

FRP Lab + Patch Oracle (experimental)

TOOL

Physics-layer survival for each FRP method, patch-timeline forecasts with calibration, and an exportable bench log.

1. It pre-fills from the connected device's patch level.
2. Read bands, not wishes: each method shows what survives and what would disprove the forecast.
3. Export the bench log JSON to calibrate claims with real measurements.

Rescue Lab (experimental) - six lanes

TOOL

PC password, carrier unlock, MiFi/modem unlock, button-phone locks, phone screen locks, and black screens - each with detection first and bands, not promises.

1. PC lane: identify the account type first (Microsoft vs local vs domain). Local accounts get the boot-USB route; Microsoft/accounts get official recovery - no software pretends otherwise.
2. Carrier lane: detect the lock type first; lender-financed phones are refused by policy with the honest route shown.
3. Modem lane: identify the exact model, read remaining attempts, then (legacy Huawei) generate verified codes or (ZTE/Alcatel) use the named $3-8 service route. Auto-Session gates every step.
4. Button-phone lane: SIM PIN vs phone lock first, then defaults (1234, 0000, 1122, 12345), then the per-brand chipset route.
5. Screen-lock lane: pick the Android era; Android <= 8 gestures can be cracked offline by the built-in tool, Android 9+ is server-side truth.
6. Black-screen lane: a yes/no triage tree - force-restart combos, then repair-shop honesty if the panel is dead.

Files

TOOL

Browse, pull and push files between computer and phone.

1. Select a device, open Files, navigate folders, use pull/push to copy.
2. Great for rescuing photos before a risky repair.

Apps

TOOL

List, inspect and manage installed packages on the device.

1. Select a device, open Apps, search for the package name.
2. Uninstall/disable actions follow normal Android permissions.

Screen

TOOL

See and control the phone's screen from the computer.

1. Select a device and open Screen.
2. Essential partner for the black-screen and screen-lock lanes when touch is broken.

Performance

TOOL

Live CPU, memory and battery readouts from the device.

1. Select a device and open Performance.
2. Use it to spot a runaway app before blaming hardware.

Logcat

TOOL

The phone's live diagnostic log stream.

1. Select a device, open Logcat, reproduce the problem, copy the errors.
2. Attach the relevant lines (never passwords) to GitHub issues.

Shell

TOOL

Direct ADB shell for experts.

1. Everything typed here runs on the phone with shell-user power - type carefully.
2. If you are not sure what a command does, do not run it; ask in Help first.

4 Myth vs physics - the two cards that stop scams

These two boxes are printed in the app's Rescue Lab too. They are the direct answer to the most common 'magic cable' and 'magic percentage' claims sold on the internet.

HDMI & magic-cable claims

SCAM ALERT

HDMI (and USB-C DisplayPort) carries PICTURE AND SOUND - nothing else. There is no command channel in it: an HDMI cable cannot remove a password, cannot bypass FRP, cannot unlock anything, on ANY device - modern or old. Any tool/video claiming an 'HDMI bypass' is a scam. What display-out REALLY does (and it's valuable): with a phone that has HDMI/DP-out (Samsung DeX, some flagships), you SEE the screen of a broken-touch phone - paired with a USB-OTG mouse you can CLICK it too. That's rescue through visibility, not bypass. Same cable law for PCs: computers take no password-reset commands through cables - the bootable USB you build IS the 'cable method'.

Where '100%' honestly lands, per device class

PER-CLASS

Where the 100% honesty lands, per device class: LEGACY Huawei modems -> the math IS deterministic for that generation (as close to 100% as physics allows - codes verified against real published examples, the risk is only attempts-management, which the app gates). Button phones -> defaults or one service-route format covers the great majority; the rest are named exceptions. Android 15/16 FRP & carrier-locked phones & Microsoft/Apple/ account laptops -> server-side: 0% by software for anyone. One number for everything would be a lie; per-class truth is the product.

5 Questions & answers (the counter favourites)

The 18 questions real users actually ask, answered with the honesty law applied. In the app these are searchable from the Help view.

Q: Can DroidKit remove FRP on Android 15/16 completely, like some apps promise?

No - and neither can they, whatever their adverts say. That decision happens on Google's server after a reset. Any tool, cable or video claiming full removal on those versions is a scam. DroidKit shows the physics, the real options per version, and the honest route (the account owner, official recovery) instead of selling the lie.

Q: Then what does '100% working' mean anywhere in this app?

Only where physics makes it true: per device class. Legacy Huawei modems - the code math is deterministic for that generation (verified against real published examples). Button phones - defaults or one known route covers the great majority. Android 15/16 FRP, carrier-locked phones, Microsoft/Apple-account laptops - zero by software, for anyone. One number for everything would be a lie; the bands page is the product.

Q: My carrier does not exist any more (Orange Kenya / Telkom-era pocket WiFi). Is my locked MiFi e-waste?

No. A MiFi checks the unlock code INSIDE itself - there is no carrier server to die. The lock is dead, not the device. The Modem lane identifies your exact model, reads remaining attempts, and gives the route: verified code generation for legacy Huawei, boot-pin firmware route for E5573Cs-609, or the named $3-8 code services for ZTE/ Alcatel. Then set the new carrier's APN (Safaricom, Airtel, Telkom, Faiba values are listed in the lane).

Q: A shop 'unlocked' my financed phone and it re-locked. Why?

Because it was never a carrier lock. Watu/M-Kopa/Lipa Mdogo Mdogo phones are lender-managed (MDM): the re-lock is the lender's server, and it will keep happening until the phone is fully paid. DroidKit refuses this job by policy - the honest route is the lender, not another shop.

Q: Can an HDMI cable bypass the lock on modern devices?

No cable can. HDMI and USB-C DisplayPort carry picture and sound only - there is no command channel in them. On ANY device, modern or old, an 'HDMI bypass' is physically impossible; anyone selling it is scamming. What display-out DOES do: with Samsung DeX-class phones you can SEE a broken-touch screen and click it with a USB-OTG mouse. Visibility, not bypass.

Q: Can a USB cable reset my laptop's password from another computer?

No - computers take no password-reset commands through any PC-to-PC cable. There is no such protocol, on any operating system. The bootable rescue USB you build IS the 'cable method'; the PC lane walks you through it, account type by account type.

Q: How many times can I try an unlock code on my MiFi?

It depends on the model - Huaweis typically allow about 10, several Alcatels hard-lock after just 3 wrong tries, and ZTEs are often around 5. This is why the Auto-Session reads your remaining attempts FIRST, blocks at 2 or fewer, validates the IMEI checksum, and lets only a human fire the final entry. Never guess a code.

Q: The V201 code generation is 'UNVERIFIED'. Why show it at all?

Because hiding it would be pretending it does not exist, and pretending it works would be worse. It is a faithful port of published Huawei work, clearly labelled, fenced off from auto-entry, and waiting for donor-bench calibration (the calibration guide is in docs/BENCH-CALIBRATION-GUIDE.md). The bench log decides graduation - not marketing.

Q: My button/keypad phone (Itel, Tecno, Nokia) asks for a phone lock code.

First make sure it is the PHONE lock, not your SIM PIN (the lane's first card separates them - SIM PIN uses defaults from the SIM pack, and PUK comes free from the carrier). For the phone lock: try the defaults 1234 / 0000 / 1122 / 12345, then follow the per-brand chipset route in the Button-phone lane. The lock lives in local firmware - no server - so this class is one of the most genuinely doable rescues in the app.

Q: I forgot my phone's pattern. Can you crack it without wiping?

Android 8 and older: yes - the Screen-lock lane includes a working gesture.key cracker (tested: all 389,112 possible patterns; it finds matches from the on-device hash). Android 9+: the lock moved into hardware-backed storage; offline cracking is over for everyone. Then the honest options are Samsung's findmymobile (if set up), or a wipe via recovery - the lane says which applies to your era.

Q: My phone fell and the screen is black but it rings. Dead?

Maybe not. The Black-screen lane is a yes/no triage: force-restart combos per brand, charge-and-try-again, screen-mirror to check if the phone is alive behind a dead panel (your data is often fully rescuable), and the honest 'panel replacement is a hardware job' verdict when it is.

Q: I forgot my Windows laptop password. What actually works?

Three different worlds: MICROSOFT account -> reset online at account.live.com/password/reset (no software can do this locally). LOCAL account -> the boot-USB route with chntpw clears it in minutes; the PC lane gives the exact steps, including the BitLocker check you must run FIRST so you do not lock yourself out of your own files. DOMAIN/ work laptop -> only your IT admin. The lane identifies which world you are in before touching anything.

Q: Will the PC rescue delete my files?

The chntpw route for local accounts does not touch personal files. 'Reset this PC' has a keep-files option, and the lane puts the data-backup step first in every route. Any route that erases is labelled ERASES before you choose it - never after.

Q: Does DroidKit send my device data anywhere?

No. It runs on your computer; there is no DroidKit account or cloud. The only file that can leave is the bench log you personally export and choose to share. The code is open if you want to verify the claim.

Q: Why is it free? What is the catch?

No catch: MIT-licensed open source. The catch exists in the $30/month tools that promise server-side-impossible miracles - the comparison lives in docs/COMPARISON-2026-FINAL.md. Where a job genuinely needs a third party (a $3-8 code service, an official recovery page) the app says so and takes no cut.

Q: Do I need internet for DroidKit to work?

Only to install it and to read linked web resources. The core work - detection, guides, code generation, the cracker, help and the PDF guide - is offline. Nothing phones home.

Q: Is unlocking my own phone or MiFi legal in Kenya?

Self-unlocking devices you own is legal. Rewriting IMEIs is illegal (Communications Authority rules) and DroidKit never touches the IMEI. Defeating lender MDM on financed phones is not unlocking - it is taking the lender's property, and this app refuses it by policy.

Q: One feature only supports my phone by talking to it over cable - why not fully automatic?

Read-only AT probes (identify model, read attempts) can be automated - the Auto-Session does this, and the native serial backend RFC is ready in docs/RFC-MODEM-SERIAL-BACKEND.md. The unlock-entry command itself stays human-fired by design: a machine must never guess against a permanent attempt counter. That is a policy, not a missing button.

6 The bands, and the words behind them

The four labels you see under every job in the app - and the small dictionary for the words repair people throw around.

DOABLE

DOABLE

Physics and the software path are on your side. The method is known, testable, and the app either does it or hands you exact steps.

Example: Legacy-Huawei MiFi NCK codes (math is deterministic for that generation), feature-phone default locks, gesture.key cracking on Android <= 8.

CONDITIONAL

CONDITIONAL

Possible, but only with the right model, the right firmware, or a named third party. Real risk exists (attempt counters, bricking, data loss) - the app gates and warns instead of guessing.

Example: E5573Cs-609 boot-pin route, ZTE/Alcatel NCK via cheap verified code services, chntpw on a Windows LOCAL account.

NOT-BY-SOFTWARE

NOT-BY-SOFTWARE

Decided on someone else's server (Google, Apple, a carrier, a lender). No app on Earth computes around that - anyone claiming otherwise is selling a lie. We name the honest route instead.

Example: Android 15/16 FRP after a hard reset, Microsoft/Apple-account laptop passwords, lender MDM locks (Watu, M-Kopa, Lipa Mdogo Mdogo).

UNVERIFIED

UNVERIFIED

A faithful port of published work that our own bench has NOT confirmed yet. It is labelled, fenced, and never auto-fired. The bench log - not marketing - decides when it graduates.

Example: Huawei V201-era NCK candidate for 2012+ models.

Small dictionary

Word	What it actually means
ADB	Android Debug Bridge - the official cable/wireless channel between computer and phone that every DroidKit phone feature uses.
FRP	Factory Reset Protection. After a reset, Android demands the previous Google account. Newer versions enforce it on Google's server.
NCK	Network Control Key - the code that removes a carrier lock from a modem/MiFi. Typed once, counted forever.
Attempt counter	How many wrong codes a lock accepts before it dies permanently. Read it before typing anything.
Luhn check	The checksum built into every IMEI; the app validates it so a typo never burns an attempt.
IMEI	The device's 15-digit identity. Reading it: fine. Rewriting it: illegal in Kenya and never touched by this app.
MDM	Mobile Device Management - remote control by a company or lender. Financed phones re-lock because of MDM, not carriers.
APN	Access Point Name - the settings entry that tells a SIM how to reach the internet (Safaricom = safaricom, Airtel = airtelgprs.com, Telkom = telkom, Faiba = jtl).

Word	What it actually means
Bootloader / EDL / Brom	Low-level chip modes used for deep repair. Powerful and brick-capable; the Rescue Lab routes you only with model-exact instructions.
EDL	Emergency Download mode (Qualcomm chips) - a fire-rescue door for dead devices, used by service tools.
Band (traffic light)	Green DOABLE / amber CONDITIONAL / red NOT-BY-SOFTWARE. The honest difficulty label on every job.
Bench log	The exportable JSON record of real measurements the lab keeps; it is how UNVERIFIED claims graduate.
Patch Oracle	The FRP Lab engine that estimates method survival against security patches and publishes falsifiable forecasts.
spd_dump	Open-source tool for Spreadtrum/Unisoc feature phones - the button-phone lane's deep route for stubborn Itel/Tecno locks.

7 Getting human help

In order, so answers are fast and free:

1. Find your exact model and Android version (System Info view, or the sticker under a MiFi's battery).
2. Search this Help view and the FAQ - most questions are already answered.
3. Check the band colour for your exact model in the FRP Lab / Rescue Lab lane.
4. Still stuck? Open a GitHub issue (link below) with: DroidKit version, device model, what you clicked, expected vs actual, and log lines from Logcat. Never include passwords or unlock codes.
5. Urgent shop counter question? The printable kid-sheets in docs/kid-sheets/ are made for exactly that moment.

Official places

What	Where
GitHub repository	https://github.com/AISACTECH/droidkitv1
Report a problem (issues)	https://github.com/AISACTECH/droidkitv1/issues
Coverage map (which brands/routes)	https://github.com/AISACTECH/droidkitv1/tree/main/docs/COVERAGE-MAP.md
Honest comparison vs paid FRP apps	https://github.com/AISACTECH/droidkitv1/tree/main/docs/COMPARISON-2026-FINAL.md
CI / install-from-git fix guide	https://github.com/AISACTECH/droidkitv1/tree/main/docs/CI-GREEN-GUIDE.md

Before you report - the 5 facts that make bugs die fast

1. Your DroidKit version (this guide covers v1.1.0).
2. The exact device model and Android version (System Info view).
3. What you clicked, in order.
4. What you expected vs what happened.
5. Relevant Logcat lines - never passwords, patterns, IMEIs of third parties, or unlock codes.

> **FINAL WORD**
>
> DroidKit will never promise you a miracle. It promises you the truth, the exact steps physics allows, and the honest route for the rest - free, offline, and in the open.